SuperChat: Phase 2 Technical Report

Real API integration · flights and shopping on public APIs · Individual submission (extends the Phase 1 report)

1. Goal and outcome

Phase 2 replaces the mock data layer with real APIs and adds authentication, rate-limit handling, pagination, latency handling, caching and logging (brief, section 4). Because Phase 1 hid all data behind repository interfaces, the work is confined to new **Api*** implementations and a shared HTTP client; the router, composer, state and the three domain modules are unchanged apart from two small hooks described in section 3. **Flights** run on the real Aviasales Data API and **shopping** on the public DummyJSON catalogue. **Cinema** stays on the mock dataset because no public cinema/showtimes API suitable for Azerbaijan could be found. The mock data remains the default (`DATA_SOURCE=json`), so the app works without any key or internet access to data providers.

2. APIs investigated and chosen

Domain	Candidate	Finding	Decision
Flights	Aviasales Data API (Travelpayouts)	Real Aviasales API, free token, endpoint `/aviasales/v3/prices_for_dates`. Returns cached fares found by Aviasales users in the last 48 h: price, airline, flight number, departure time, transfers, duration. No baggage, seats left, arrival time or booking.	**Used**
Shopping	Birmarket (the marketplace named in the brief)	No public API found.	Not possible
Shopping	DummyJSON `/products`	Public demo e-commerce API, no key, limit/skip pagination, 194 products with price, stock, brand, category, rating. English names only; no colour.	**Used** as stand-in
Cinema	CinemaPlus / local cinemas	No public API found.	Not possible
Cinema	International Showtimes, Film Chase	Showtimes APIs, but paid (about 149 EUR/month; free tier only a 7-day trial or 50 requests/month, UK focus). Azerbaijani coverage not confirmed.	Not used
Cinema	TMDB	Free film metadata but no cinemas or showtimes; mixing it with mock showtimes would require inventing showtimes.	Not used

3. Design

Per-domain source. `build_repositories()` picks the implementation per domain from `FLIGHTS_SOURCE` / `SHOPPING_SOURCE` (default `DATA_SOURCE`). A misconfigured real source (for example a missing token) logs an error and falls back to the mock data instead of crashing the app. Cinema always uses the mock repositories.

Shared HTTP client (`api_client.py`): used by both adapters; adapters only map fields. **Adapters** (`api_repositories.py`): ApiFlightRepository and ApiProductRepository convert API records into the exact record shape of the mock dataset (Appendix A), so modules need no knowledge of the source.

Two changes outside the data layer. (1) A no-op `FlightRepository.prepare(origin, destination, date)` hook that flight search calls first: a real flight API must be queried per route, while the mock repository already holds everything. (2) Flights and the composer tolerate fields a real source does not provide (`None` is shown as “not available”, the LLM is told never to guess them).

Honesty of data. Missing fields are never invented: arrival time, baggage and seats left are empty for real flights; AZ/RU product names equal the English title; prices are USD converted to AZN at a configurable rate (`USD_AZN_RATE`, default 1.70, the long-standing peg). The Aviasales API has no booking endpoint, so a flight reservation remains a local mock (allowed by the brief). If a route/date has no cached fare, the assistant says nothing was found. City names are mapped to IATA codes by a built-in directory (`CITY_CODES`).

4. Phase 2 requirements and how they are met

Requirement	Implementation
Authentication	Aviasales token sent in the `X-Access-Token` header, read from `.env` / the hosting environment; never written to the repository, logs or error messages. DummyJSON needs no key.
Rate limits	Retry on HTTP 429/5xx and network errors with exponential back-off; honours `Retry-After` (capped at 5 s); optional client-side limit `API_MAX_RPS`; results are cached so repeated questions do not hit the API.
Pagination	DummyJSON: limit/skip loop until `total` is reached (two requests for 194 products). Aviasales: `limit` + page (30 cheapest fares per query).
Latency	Per-request timeout (5 s, `API_TIMEOUT`), at most 3 attempts; if an API is down the user gets a one-sentence message and the chat keeps working.

Requirement	Implementation
Caching	TTL cache (5 min, `API_CACHE_TTL`) per query; stale data is served when a refresh fails; earlier flight results stay available so “back to the flights” still works.
Logging	Method, path, status and duration of every call, retry warnings, stale-cache warnings; secrets excluded.
Errors	Data-source errors never reach the user as stack traces (brief 3.7): flight search returns a friendly error result; other failures are caught per intent in the engine.

5. Testing

25 pytest tests pass: the 11 Phase 1 module tests plus 14 new tests in `tests/test_api.py`. They run against in-process mock servers (`httpx.MockTransport`) whose responses copy the documented Travelpayouts and DummyJSON formats, and check: field mapping and USD→AZN conversion, correct endpoint/parameters and token header, no request for an unknown city, caching, pagination order, retry after 429, a rejected token producing a clean error without leaking it, stale cache during an outage, a full chat that survives a total outage, shopping and cart on API products (stock from the API), cinema remaining on mock data, fallback to mock flights without a token, and per-domain source selection. The Phase 1 routing evaluation is unaffected (40 conversations; LLM 40/40, rules-only 38/40). **Live calls to the two real APIs are not part of the automated tests** and must be verified manually with a Travelpayouts token (see README).

6. Known limitations

• Aviasales returns cached fares only: some routes/dates are empty, and baggage, seats and arrival time are unavailable; flights can be searched only for cities in `CITY_CODES`. • DummyJSON is a demo catalogue, not Birmarket: English names, no colours, no real purchasing (cart is local). • Cinema has no real API: films, cinemas and showtimes are mock data. • Filtering still happens in the modules after fetching a list; a very large catalogue would need server-side search in the repository. • Sessions and mock reservations are in memory.

7. Sources and licences

Aviasales Data API (Travelpayouts, `support.travelpayouts.com`), used under its partner terms with a personal free token; DummyJSON (`dummyjson.com`, free public test API); httpx (BSD), pytest (MIT), FastAPI (MIT). No personal or payment data is used; reservations are mocks.